

Basic Query, Aggregate Functions & Grouping Data

PTP Program





Training Guidelines

Untuk mengikuti materi pelatihan berikut ada beberapa hal yang harus diperhatikan oleh peserta:

- Peserta diharapkan sudah memiliki pengetahuan dasar tentang **database** dan **SQL**.
- Peserta diharapkan memperhatikan materi dan mencatat materi atau penjelasan yang disampaikan dari instruktur.
- Instruktur diharapkan dapat menjelaskan materi dengan jelas dan memastikan peserta pelatihan dapat memahami topik yang dibahas.
- Untuk contoh latihan dan kode atau syntax serta beberapa penjelasannya, anda dapat melihat di **External References** yang ada pada **slide terakhir**.



Point of Discussion

- Memahami kegunaan dan perintah DQL
- Memahami Pernyataan Kondisi dalam SQL
- Memahami Cara Menggunakan Grouping dan Aggregate Functions
- Memahami Cara Membuat Index dan Explain Query



DQL (Data Query Language)

Data Query Language

Apa itu DQL?

DQL (Data Query Language) adalah bagian dari SQL yang digunakan untuk memanipulasi dan mengambil data dari basis data. DQL berfokus pada query atau perintah untuk membaca data dan menampilkan hasilnya tanpa memodifikasi data di dalam database. Dalam DQL terdapat beberapa Klausa yang sering digunakan, diantaranya:

- WHERE
- ORDER BY
- LIMIT

Contoh Syntax DQL:

```
SELECT column1, column2
FROM table_name
WHERE condition
ORDER BY column1 ASC
LIMIT n;
```

Silahkan download file [dql_stat.sql](#) dan jalankan query tersebut untuk membuat tabel **teachers** dan **employees** yang nantinya digunakan dalam beberapa contoh query yang ada pada topik **DQL, Grouping & Aggregation**.

Data Query Language

DISTINCT

Dalam sebuah tabel, bukan hal yang aneh jika sebuah kolom berisi baris dengan nilai duplikat., Kita dapat menggunakan kata kunci **DISTINCT** sebagai bagian dari kueri yang menghilangkan duplikat dan hanya menampilkan nilai unik. Gunakan kata kunci **DISTINCT** segera setelah **SELECT**.

```
-- Example  
SELECT DISTINCT School FROM teachers
```

Perintah ini juga berfungsi pada lebih dari satu kolom pada satu waktu. Jika kita menambahkan kolom, kueri akan mengembalikan setiap pasangan nilai yang unik.

```
-- Example  
SELECT DISTINCT School,Salary  
FROM teachers
```

Conditional

Klausu WHERE

Klausu **WHERE** digunakan untuk memfilter data yang diambil berdasarkan kondisi tertentu. Kata kunci WHERE memungkinkan Anda menemukan baris yang cocok dengan nilai tertentu, rentang nilai, atau beberapa nilai berdasarkan kriteria yang diberikan melalui operator. Anda juga dapat mengecualikan baris berdasarkan kriteria. Berikut adalah Syntax-nya:

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition;
```

```
-- Example  
SELECT last_name, school, hire_date  
FROM teachers  
WHERE school = 'Myers Middle School';
```

Conditional

Comparison Operators

Dalam SQL, Anda dapat menggunakan operator lain selain operator **=** untuk memfilter data berdasarkan kondisi atau melakukan komparasi.

Operator	Function	Example
=	Equal to	WHERE school = 'Baker Middle'
<> or !=	Not equal to	WHERE school <> 'Baker Middle'
>	Greater than	WHERE salary > 20000
<	Less than	WHERE salary < 60500
>=	Greater than or equal to	WHERE salary >= 20000
<=	Less than or equal to	WHERE salary <= 60500
BETWEEN	Within a range	WHERE salary BETWEEN 20000 AND 40000
IN	Match one of a set value	WHERE last_name IN ('Bush','Roush')
LIKE	Match a pattern (case sensitive)	WHERE first_name LIKE 'Sam%'
ILIKE	Match a pattern (case sensitive)	WHERE first_name ILIKE 'sam%'
NOT	Negates a condition	WHERE first_name NOT ILIKE 'sam%'

Conditional

AND vs OR

Kita dapat menggunakan operator AND dan OR untuk memfilter data berdasarkan kondisi tertentu.

- Operator **AND** menampilkan rekaman atau data jika **semua** kondisinya BENAR.
- Operator **OR** menampilkan rekaman atau data jika **salah satu** kondisi bernilai BENAR.

Catatan: Anda tidak dapat menggunakan AND untuk kolom yang sama namun kondisinya berbeda. Misalkan anda ingin **last_name** 'Cole' dan 'Bush', anda tidak dapat menggunakan AND, karena ini berarti **last_name** harus bernilai 'Cole' dan 'Bush' pada saat yang sama, yang tidak mungkin terjadi dalam satu baris. Pada kasus ini anda bisa gunakan **OR** atau **WHERE last_name IN ('Cole', 'Bush')**

```
-- Example 1
SELECT *
FROM teachers
WHERE school = 'Myers Middle School'
AND salary < 40000;
```

```
-- Example 2
SELECT *
FROM teachers
WHERE last_name = 'Cole'
OR last_name = 'Bush';
```

```
-- Example 3
SELECT *
FROM teachers
WHERE school = 'F.D. Roosevelt HS'
AND (salary < 38000 OR salary > 40000);
```

Sorting

ORDER BY Command

Dalam SQL, kita dapat mengurutkan hasil query menggunakan klausa **ORDER BY** diikuti dengan nama kolom atau kolom yang akan diurutkan.

Secara default, ORDER BY mengurutkan nilai dalam urutan menaik atau disebut dengan *Ascending* (ASC), tetapi untuk mengurutkan dalam urutan menurun (*Descending*), anda dapat melakukan dengan menggunakan kata kunci DESC.

```
-- Example
SELECT first_name, last_name, salary
FROM teachers
ORDER BY salary DESC;
```



Grouping and Aggregation

Grouping & Aggregation

GROUP BY Command

Klausu GROUP BY digunakan dalam kombinasi dengan fungsi agregat untuk mengelompokkan baris berdasarkan satu atau beberapa kolom dan melakukan perhitungan pada setiap kelompok. Klausu GROUP BY membagi kumpulan hasil ke dalam kelompok, dan fungsi agregat beroperasi pada setiap kelompok secara terpisah.

Berikut sintaks dasar query GROUP BY dalam SQL:

```
SELECT column1, column2, aggregate_function(column3)
FROM table_name
WHERE condition
GROUP BY column1, column2;
```

Perhatikan bahwa saat menggunakan klausu GROUP BY, pernyataan SELECT harus menyertakan kolom yang digunakan untuk pengelompokan atau fungsi agregat yang diterapkan pada kolom tersebut. Kolom non-agregat lainnya tidak boleh disertakan dalam pernyataan SELECT kecuali kolom tersebut juga merupakan bagian dari klausu GROUP BY.

Grouping & Aggregation

Aggregate Functions

Pernyataan GROUP BY sering digunakan dengan fungsi agregat (COUNT(), MAX(), MIN(), SUM(), AVG()) untuk mengelompokkan kumpulan hasil berdasarkan satu atau lebih kolom.

```
SELECT department, COUNT(salary) FROM  
employees GROUP BY department;
```

```
SELECT department, SUM(salary) FROM employees  
GROUP BY department;
```

```
SELECT department, AVG(salary) FROM employees  
GROUP BY department;
```

```
SELECT department, MIN(salary) FROM employees  
GROUP BY department;
```

```
SELECT department, MAX(salary) FROM employees  
GROUP BY department;
```

- **COUNT:** Mengembalikan jumlah baris yang sesuai dengan kondisi tertentu atau jumlah total baris dalam tabel.
- **SUM:** Menghitung jumlah kolom numerik.
- **AVG:** Menghitung nilai rata-rata kolom numerik.
- **MIN:** Mengembalikan nilai minimum kolom.
- **MAX:** Mengembalikan nilai maksimum kolom.

Grouping & Aggregation

HAVING

Klausula **HAVING** digunakan untuk memfilter hasil kueri yang menyertakan klausula **GROUP BY**. Klausula ini memungkinkan Anda menentukan kondisi untuk pengelompokan berdasarkan fungsi agregat. Klausula **HAVING** mirip dengan klausula **WHERE**, tetapi beroperasi pada data yang dikelompokkan, bukan baris individual. Berikut adalah Syntax dan contoh penerapannya:

```
SELECT column1, aggregate_function(column2)
FROM table_name
WHERE condition
GROUP BY column1
HAVING condition;
```

```
SELECT department, AVG(salary) as average_salary
FROM employees
GROUP BY department
HAVING AVG(salary) > 50000;
```

Perhatikan bahwa klausula **HAVING** diterapkan setelah pengelompokan terjadi, jadi Anda dapat menggunakan fungsi agregat dalam klausula **HAVING** untuk memfilter grup berdasarkan nilai agregatnya. Klausula **HAVING** khususnya berguna saat Anda ingin memfilter grup berdasarkan kondisi agregat yang tidak dapat dinyatakan dalam klausula **WHERE**.



Index

Database Indexing

Apa itu Database Indexing?

Bayangkan data yang Anda miliki dalam sebuah database dianalogikan sebagai **buku-buku yang disimpan di dalam sebuah perpustakaan**. Lalu bayangkan jika **Anda diminta untuk mengambil buku tertentu di dalam perpustakaan tersebut**. Bagaimana Anda bisa menemukan buku yang diminta?

Jika perpustakaannya kecil, Anda bisa menemukannya dengan mudah, namun **bagaimana jika perpustakaannya sangat besar? Apakah Anda akan mencarinya satu per satu?**



Source:

<https://www.dreamstime.com/illustration/kids-library.html>

Database Indexing

Apa itu Database Indexing?

Di beberapa perpustakaan, biasanya akan ditemukan sebuah **buku index** yang memuat kode buku serta lokasi penempatan pada perpustakaan.

Dengan buku index, Anda bisa menemukan buku menjadi lebih mudah.

Ketika anda membaca buku juga terkadang terdapat juga Index yang memudahkan kita untuk mencari lokasi halaman untuk sebuah kata kunci yang ingin kita cari.



Database Indexing

Apa itu Database Indexing?

Hal tersebutlah yang bisa kita analogikan sebagai indeks. Secara formal, **database index** adalah **sebuah struktur data yang digunakan untuk memetakan data di dalam sebuah database.**

Berikut ini adalah kegunaan dari Index dalam database:

1. Indeks mempercepat operasi SELECT, WHERE, JOIN, dan ORDER BY yang dapat membantu dalam menemukan data lebih cepat.
2. Tanpa indeks, pencarian data pada kolom akan mengharuskan MySQL untuk memindai seluruh tabel (full table scan), yang bisa sangat lambat terutama pada tabel besar.

Database Indexing

Cara Kerja Menampilkan Data

Terdapat 3 cara bagaimana database mengambil data

1. Melakukan *full scan* pada tabel. Ini adalah cara paling mahal yang dilakukan oleh database namun cara ini biasanya digunakan untuk tabel yang kecil.
2. Akan mengambil data lewat indeks dan hasilnya akan digunakan untuk filter ke tabel yang sebenarnya
3. Semua data akan diambil lewat indeks.

Dari ketiga cara tersebut menggunakan indeks dapat mempercepat kita untuk melakukan proses pencarian data dibandingkan dengan full scan.

Database Indexing

Cara Membuat Index

Sintaks Dasar:

```
CREATE INDEX index_name ON table_name(column_name);
```

Contoh:

Misalkan Anda ingin membuat indeks pada kolom `city` di tabel `customers`:

```
CREATE INDEX idx_city ON customers(city);
```

Jika Anda ingin membuat composite index (indeks gabungan) pada beberapa kolom, misalnya pada kolom `city` dan `country`:

```
CREATE INDEX idx_city_country ON customers(city, country);
```

Database Indexing

Cara Membuat Index

Untuk melihat daftar indeks yang ada pada tabel, gunakan perintah `SHOW INDEX`.

Sintaks Dasar:

```
SHOW INDEX FROM table_name;
```

Contoh:

Misalkan Anda ingin melihat semua indeks yang ada di tabel `customers`:

```
SHOW INDEX FROM customers;
```

Database Indexing

Cara Membuat Index

Jika indeks tidak lagi dibutuhkan atau ingin dihapus, gunakan perintah **DROP INDEX**.

Sintaks Dasar:

```
DROP INDEX index_name ON table_name;
```

Contoh:

Misalkan Anda ingin menghapus indeks `idx_city` pada tabel `customers`:

```
DROP                INDEX                idx_city                ON                customers;
```

Catatan: Untuk menghapus primary key, gunakan **ALTER TABLE** seperti ini:

```
ALTER TABLE customers DROP PRIMARY KEY;
```



Explain Query

Explain Query

Kegunaan Explain Query

Setiap kueri SQL pada dasarnya tidak terkait dengan kinerja, tetapi ketika digunakan dalam data yang besar, masalah kinerja selalu ada. Untuk mengatasi masalah ini, satu-satunya solusi yang mungkin adalah kueri tersebut harus menentukan apa yang sedang dilakukannya dan berapa banyak waktu yang dihabiskannya untuk melakukan tugas tersebut. Dengan bantuan **EXPLAIN Query** analis data dapat mengetahui hal tersebut, mereka mungkin dapat **mengoptimalkan kueri** tersebut secara maksimal.

EXPLAIN Query adalah perintah dalam SQL yang digunakan untuk menganalisis dan memahami bagaimana sebuah query dieksekusi oleh sistem basis data. Dengan menggunakan EXPLAIN, pengguna dapat melihat langkah-langkah yang dilakukan oleh database untuk menjalankan query tertentu, seperti membaca tabel, menggunakan indeks, atau melakukan join.

Hal utama yang perlu diperhatikan tentang EXPLAIN adalah bahwa ia akan digunakan di awal kueri, yaitu sebelum SELECT, INSERT, UPDATE, dan seterusnya.

```
EXPLAIN query;
```


Explain Query

Penggunaan Explain

Kita dapat menyimpulkan bahwa tujuan Penggunaan EXPLAIN Query adalah sebagai berikut:

- **Analisis Performa Query:** Memahami bagaimana query dieksekusi untuk mengidentifikasi potensi hambatan kinerja.
- **Optimasi Query:** Menemukan cara untuk mempercepat query, seperti menambahkan indeks atau memodifikasi query.
- **Debugging:** Memeriksa query untuk memastikan tidak ada kesalahan atau eksekusi yang tidak efisien.

Misalkan kita memiliki tabel customers, di sini kita akan melakukan EXPLAIN pada query di bawah ini :

```
SELECT * FROM customers  
WHERE city = 'New York';
```

Kita bisa menggunakan EXPLAIN dengan cara berikut:

```
EXPLAIN SELECT * FROM  
customers WHERE city =  
'New York';
```

Explain Query

Penggunaan Explain

Hasil EXPLAIN bisa dilihat pada gambar di bawah ini, beserta interpretasi dari hasil

EXPLAIN SELECT * FROM customers WHERE city = 'New York';

id	select_ty...	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶ 1	SIMPLE	customers	NULL	ALL	NULL	NULL	NULL	NULL	122	10.00	Using where

Interpretasi Kolom

- **Id:** Nilainya 1, yang menunjukkan bahwa ini adalah query sederhana dengan satu langkah pemrosesan, tanpa subquery atau join yang kompleks.
- **select_type:** Menunjukkan SIMPLE, yang berarti query ini adalah query dasar, bukan subquery atau join.
- **table:** Nama tabel yang digunakan dalam query, yaitu customers.
- **partitions:** Nilainya NULL, yang menunjukkan bahwa partisi tidak digunakan dalam tabel customers. Jika tabel mendukung partisi dan query melibatkan partisi tertentu, ini akan menunjukkan partisi yang digunakan.

Explain Query

Penggunaan Explain

	id	select_ty...	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
►	1	SIMPLE	customers	NULL	ALL	NULL	NULL	NULL	NULL	122	10.00	Using where

Interpretasi Kolom (Lanjutan)

- **type:** Menunjukkan **ALL**, yang merupakan **jenis akses data yang paling tidak efisien** karena berarti MySQL harus melakukan full table scan (memindai seluruh tabel) untuk menemukan data yang cocok. **Hal ini menunjukkan bahwa tidak ada indeks yang digunakan untuk mempercepat pencarian dalam query ini.** Jika indeks pada kolom city tersedia dan digunakan, seharusnya MySQL menampilkan jenis akses seperti ref atau index, yang lebih efisien.
- **Possible_keys:** Nilainya NULL, yang menunjukkan bahwa MySQL tidak menemukan indeks yang sesuai atau tidak ada indeks yang tersedia untuk digunakan pada kolom city dalam kondisi WHERE.
- **key:** Nilainya NULL, yang berarti MySQL tidak menggunakan indeks apa pun untuk mengeksekusi query ini.

Explain Query

Penggunaan Explain

	id	select_ty...	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
►	1	SIMPLE	customers	NULL	ALL	NULL	NULL	NULL	NULL	122	10.00	Using where

Interpretasi Kolom (Lanjutan)

- **Key_len**: Nilainya NULL, karena tidak ada indeks yang digunakan. key_len menunjukkan panjang indeks yang digunakan dalam byte, tetapi dalam kasus ini, kolom tersebut kosong.
- **ref**: Nilainya NULL, karena tidak ada indeks yang digunakan untuk menghubungkan tabel dengan nilai tertentu.
- **rows**: Menunjukkan 122, yang berarti MySQL memperkirakan harus memindai sekitar 122 baris dalam tabel customers untuk menemukan hasil. Ini bisa berarti bahwa tabel customers memiliki sekitar 122 baris, dan MySQL harus membaca semua baris ini karena tidak ada indeks yang mempercepat pencarian.

Explain Query

Penggunaan Explain

	id	select_ty...	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
►	1	SIMPLE	customers	NULL	ALL	NULL	NULL	NULL	NULL	122	10.00	Using where

Interpretasi Kolom (Lanjutan)

- **filtered:** Menunjukkan 10.00, yang berarti MySQL memperkirakan bahwa sekitar 10% dari baris dalam tabel customers akan sesuai dengan kondisi WHERE (city = 'New York').
- **Extra:** Menunjukkan **Using where**, yang berarti bahwa MySQL harus menggunakan kondisi **WHERE** untuk memfilter hasil setelah memindai tabel. **Tidak ada indikasi penggunaan indeks di sini.**

Jika anda ingin melihat dokumentasi lengkap tentang Explain di MySQL anda dapat melihat [disini](#)

Explain Query

Penggunaan Explain Analyze

EXPLAIN ANALYZE adalah perintah yang lebih rinci dari EXPLAIN standar dan mulai tersedia di MySQL 8.0.19. Perintah ini memberikan informasi yang lebih mendalam tentang bagaimana query benar-benar dieksekusi, termasuk waktu eksekusi dan jumlah baris yang diproses secara aktual oleh setiap langkah dalam query plan.

Berikut adalah perbedaan utama antara EXPLAIN dan EXPLAIN ANALYZE:

- EXPLAIN: Memberikan rencana eksekusi teoretis, yang memperkirakan berapa banyak baris yang akan diproses dan indeks apa yang akan digunakan.
- EXPLAIN ANALYZE: Memberikan **informasi real-time**, termasuk **waktu eksekusi yang sebenarnya** dan jumlah baris yang benar-benar diproses untuk setiap langkah dalam query. Ini sangat berguna untuk menganalisis performa query secara lebih akurat.

Explain Query

Penggunaan Explain Analyze

Contoh Query

```
EXPLAIN ANALYZE SELECT * FROM customers;
```

Result Query

```
EXPLAIN
```

```
-> Table scan on customers (cost=12.70 rows=122) (actual time=0.123..0.219 rows=122 loops=1)
```

Notes: EXPLAIN ANALYZE akan benar-benar menjalankan query yang kita tulis dan EXPLAIN ANALYZE memakan cost yang cukup besar, sehingga jika ingin menjalankan EXPLAIN ANALYZE lebih baik tidak sering-sering digunakan.



External References

DQL, Grouping ,
Aggregate
Function, Index
& Explain Query

[Visit Here](#)



Hacktiv8id



Hacktiv8id



Hacktiv8



Hacktiv8 Indonesia

